

Универзитетски проект
Софтверски квалитет и тестирање

Документација за тестирање на софтвер

Volter — ERP систем за ланец на заложни куќи

Изработил:

Борјан Ѓорѓиевски

Индекс: 231175

Тема на проектот:

Volter — ERP систем за ланец на заложни куќи (тестирање на PawnService —
сервисот за залог)

Репозиториум (source code + тестови):

<https://github.com/80rjan/Volter>

- Код што се тестира (source): backend/src/main/java/com/volter/...
- Код на тестовите: backend/src/test/java/skit_project/tests/...

1. Вовед

Овој документ ја опишува стратегијата и имплементацијата на тестирањето на модулот за залог (pawn) од системот Volter — ERP систем за ланец на заложни куќи. Целниот предмет под тест е апликацискиот сервис PawnService и придружните доменски класи (PawnContract, Money), заедно со неговиот REST контролер (PawnController).

Тестирањето е организирано по техники обработени на предметот, при што секоја техника е сместена во своја папка под backend/src/test/java/skit_project/tests/, со посебен тест-код и (каде што постои) придружно текстуално објаснување. Дополнително се вклучени и техники надвор од материјалот на предметот (Spring Boot slice и end-to-end тестирање, API тестирање преку Postman, и тестирање базирано на својства).

Опфатени техники:

- Партиционирање на влезен простор (ISP) — Base Choice Coverage (BCC)
- Тестирање базирано на графови — All-du-paths, Edge-Pair, Prime Path
- Логичко тестирање — GACC / CACC / RACC
- Тестирање базирано на синтакса — мутациско тестирање со реални мутанти (PIT)
- Mockito + JUnit — тестирање на интеракции
- Spring Boot @WebMvcTest — slice тестирање на контролерот (дополнителна техника)
- API тестирање преку Postman — црна кутија врз реален HTTP (дополнителна техника)
- Spring Boot end-to-end — интеграциски тест на целосен работен тек (дополнителна техника)
- Тестирање базирано на својства (Property-based) — дополнителна техника

Намерно НЕ е опфатено во оваа фаза:

- UI (end-to-end кон прелистувач) тестирање — не е потребно во оваа фаза.

2. Алатки и технологии

Проектот е Spring Boot апликација напишана во Java, со Maven како систем за градење. Подолу се сите алатки/технологии користени за тестирањето и деталите за нивното користење.

2.1. Основа

- **Java / Spring Boot:** Java 21 и Spring Boot 4.0.3 — јазик и апликациски фрејмворк на системот што се тестира.
- **Maven:** систем за градење; тестовите се извршуваат преку Maven Surefire (unit) и Failsafe (интеграциски).
- **Бази:** PostgreSQL во продукција/интеграциски тестови; H2 (in-memory) за unit тестови.

2.2. Рамки за тестирање

- **JUnit 5:** (Jupiter) — основната рамка за тестирање: @Test, @DisplayName, @RepeatedTest, @BeforeEach/@AfterEach.

- **Mockito:** рамка за мокирање: @Mock, @InjectMocks, when/thenReturn, verify, ArgumentCaptor, InOrder, строго стубирање (MockitoExtension).
- **AssertJ:** флуентни асертации (assertThat(...).isEqualTo(...), anySatisfy) во интеграциските тестови.

2.3. Spring Boot поддршка за тестирање

- **@WebMvcTest (slice):** подига само веб-слојот (контролер + JSON + валидација + @ControllerAdvice) со мокиран сервис.
- **MockMvc:** извршување на HTTP барања во-процес, без вистински сервер/сокет.
- **@MockitoBean:** замена на Spring bean со Mockito мок (во Spring Boot 4 го заменува стариот @MockBean).
- **@SpringBootTest:** подига цела апликација за end-to-end интеграциски тест.

2.4. Специјализирани алатки

- **PIT (pitest):** алатка за мутациско тестирање; ги генерира и извршува РЕАЛНИТЕ мутанти врз бајткодот и го пресметува мутацискиот резултат. Се вклучува преку Maven профил `pit`.
- **Testcontainers:** подигнува вистински PostgreSQL во контејнер за интеграциските тестови (со fallback кон надворешна база преку IT_DATASOURCE_URL).

2.5. Организација и извршување

Секоја техника е во своја папка под src/test/java/skit_project/tests/:

isp/	графови се тестираат со ISP (BCC)
graph/	Graph coverage (all-du-paths, edge-pair, prime path)
logic/	GACC / CACC / RACC (два предиката)
syntax/	мутациско тестирање (PIT)
mockito_junit/	Mockito + JUnit
web_mvc/	Spring Boot @WebMvcTest
postman/	API тестирање преку Postman (колекција за увоз)
workflow/	Spring Boot end-to-end интеграциски тест
property_based/	тестирање базирано на својства

Извршување:

```
./mvnw test -Dtest='skit_project.tests.**,!*IntegrationTest' # unit техники
./mvnw -Ppit test-compile org.pitest:pitest-maven:mutationCoverage # мутациско
./mvnw verify # + интеграциски (бара
```

Docker)

Забелешка за градењето:

Лабораториските поднесоци под skit_project/example_labs се самостојни под-проекти (Selenium, JUnit 3, свои build фајлови) и се исклучени од Maven test компилацијата (pom.xml: compiler <testExcludes> и surefire <excludes>). Тестовите во skit_project/tests се обични тестови на реалната апликација.

3. Партиционирање на влезен простор (ISP) — Base Choice Coverage

Единица под тест:

PawnService.redeem(Long, PawnRedeemRequest, Long) — откуп на залог.

Тест: *isp/PawnRedeemIspTest.java*

3.1. Техника и алатки

ISP го дели влезниот простор на карактеристики (интерфејсни — од аргументите; функционални — од тоа што прави функцијата), секоја поделена на дисјунктни и комплетни блокови. Со Base Choice Coverage (BCC) се избира еден базен избор, па во секој тест се менува по еден блок додека останатите се држат на базната вредност.

Алатки: JUnit 5 + Mockito (сите соработници на сервисот се мокирани; се проверува исходот и интеракциите).

3.2. Карактеристики и партиции

```
interface:
  IC1: contractId постои                -> {T, F}
  IC2: cashRegisterSessionId е за отворена сесија -> {T, F}
functional:
  FC1: платено во однос на очекувано      -> {помал (LT), еднаков (E), поголем (GT)}
  FC2: вработениот има менаџер           -> {T, F}
```

Сите карактеристики се дисјунктни (блоковите не се преклопуваат) и комплетни (го покриваат секој можен влез).

3.3. BCC — базен избор и тест-множество

Базен избор: IC1=T, IC2=T, FC1=еднаков, FC2=има менаџер.

базен	IC1	IC2	FC1	FC2	Исход	Тест метод
base_paidEqualsExpected_noFlag	T	T	еднаков	T	откуп ОК, без знаменце	
IC1->F	F	T	(н/д)	T	ResourceNotFoundException	
ic1_missingContract_throws	T	F	(н/д)	T	BusinessRuleException (нема сесија)	
ic2_sessionNotOpen_throws	T	T	помал	T	потплата -> RISK_FLAG до менаџер	
fc1_underpaid_withManager_raisesRiskFlag	T	T	поголем	T	преплата, без знаменце	
fc1_overpaid_noFlag	T	T	(помал)*	F	потплата, но никој не е известен	
fc2_underpaid_noManager_noNotification						

(*) Забелешка за изводливост (FC2→F): со базниот FC1=еднаково, гранката за менаџер воопшто не се извршува, па FC2 нема забележлив ефект. Затоа за тој ред FC1 се менува на „помал“ за ефектот на FC2 да стане набљудлив (комбинацијата и понатаму е изводлива, само не и информативна со базниот блок). Вкупно: 6 теста.

4. Тестирање базирано на графови (Graph Coverage)

Единица под тест:

PawnTxQueryService.findCustomerNamesByTransactionIds(Collection<Long>) — мапира ид на трансакции во имиња на купувачи.

Тест: *graph/PawnCustomerNamesGraphTest.java*

4.1. Техника и алатки

Од контролниот тек на функцијата (CFG) се изведуваат тест-барања за три критериуми: All-du-paths (по дефиниција/употреба на променлива), Edge-Pair (парови соседни гранки) и Prime Path (максимални прости патеки). Истите 4 теста (различни влезни колекции) ги обиколуваат сите три критериуми. Алатки: JUnit 5 + Mockito (репозиториумот е мокиран за да врати 0/1/2 реда).

4.2. Контролен тек (CFG)

```
Јазли:
1 : if (transactionIds.isEmpty())
2 : return Map.of() (завршен)
3 : names = new HashMap<>(); rows = repo.find(...)
4 : услов на јамка (има ли следен ред?)
5 : names.put(row[0], row[1])
6 : return names (завршен)
Гранки: (1,2) (1,3) (3,4) (4,5) (5,4)back (4,6)
Почетен: 1 Завршни: {2, 6}
```

4.3. Тест-барања и множество

```
PRIME PATHS:
pp1 [1,2] pp2 [1,3,4,6] pp3 [1,3,4,5]
pp4 [5,4,6] pp5 [4,5,4] pp6 [5,4,5]
EDGE-PAIRS: ep1..ep7 (1,2 / 1,3,4 / 3,4,5 / 3,4,6 / 4,5,4 / 5,4,5 / 5,4,6)
ALL-DU-PATHS за `names`: du1 [3,4,5] (def@3, use@5), du2 [3,4,6] (def@3, use@6)
```

```
Тестови (патеки):
A : 1,2 (празен влез) -> pp1, ep1
B : 1,3,4,5,4,6 (еден ред) -> pp3/pp4/pp5, du1
C : 1,3,4,5,4,5,4,6 (два реда) -> pp6, втора итерација
D : 1,3,4,6 (непразен, 0 реда) -> pp2, du2
=> множеството {A,B,C,D} ги задоволува сите три критериуми.
```

Prime Path покриеноста ја вклучува (subsumes) Edge-Pair, а таа Edge и Node покриеноста, па истото множество ги задоволува сите.

5. Логичко тестирање (GACC / CACC / RACC)

Опфатени се ДВА предиката за да се покаже разликата меѓу критериумите:

- 2-клаузен (конјункција) каде GACC = CACC = RACC — logic/GoldFilterLogicTest.java
- 3-клаузен мешан а && (b || c) каде RACC е строго посилен — logic/ReportVisibilityLogicTest.java

5.1. Техника

За секоја клауза се бара таа да биде „активна“ (major) — да ја определува вредноста на предикатот. GACC: главната клауза е и точна и неточна, споредните може да се разликуваат. CACC: дополнително предикатот мора да добие различни вредности. RACC: споредните клаузи мора да бидат ИДЕНТИЧНИ во двата реда. Алатки: JUnit 5 + Mockito.

5.2. Предикат 1 — филтер за злато (P = a && b)

Во PawnService.summarize: a = (item != null), b = (item.getType() == GOLD).

ред	a	b	P=a&&b	a акт.?	b акт.?
1	T	T	T	да	да
2	T	F	F	не	да
3	F	T	F	да	не
4	F	F	F	не	не

Кај чиста конјункција секоја клауза е активна само кога другата е ТОЧНА, па:

GACC = CACC = RACC = { ред1, ред2, ред3 }

Тестови: goldItem_predicateTrue_isCounted (ред1), nonGoldItem...(ред2), nullItem...(ред3),
+ mixedSet_onlyGoldCounted (сите три одеднаш).

5.3. Предикат 2 — видливост на извештај (P = a && (b || c))

Во ReportService.get: a = извештајот е во продавницата на повикувачот; b = системски извештај (без сопственик); c = субјектот е повикувачот или негов подреден.

ред	a	b	c	b c	P	a акт.?	b акт.?	c акт.?
r1	T	T	T	T	T	да	не	не
r2	T	T	F	T	T	да	да	не
r3	T	F	T	T	T	да	не	да
r4	T	F	F	F	F	не	да	да
r5	F	T	T	T	F	да	не	не
r6	F	T	F	T	F	да	не	не
r7	F	F	T	T	F	да	не	не
r8	F	F	F	F	F	не	не	не

Клауза b: единствен пар {r2,r4} -> GACC=CACC=RACC

Клауза c: единствен пар {r3,r4} -> GACC=CACC=RACC

Клауза a (активна кога b||c = точно):

RACC (исти споредни): {r1,r5}, {r2,r6}, {r3,r7}

-> 3 пара

GACC/CACC (споредни може да се разликуваат): сите 9 комбинации

-> 9 пара

=> RACC е СТРОГО ПОСИЛЕН (RACC ⊆ GACC = CACC).

Избрано RACC тест-множество { r2, r3, r4, r6 } (мапирано на тестовите

r2_true_inShopSystemReport_returned, r3_true_subjectIsSubordinate_returned,

r4_false_subjectNotVisible_denied, r6_false_wrongShop_denied).

6. Тестирање базирано на синтакса — мутациско тестирање (PIT)

Единица под тест:

PawnContract.totalDue(), expectedRedemptionAmount(), isMatured() — аритметиката на откуп.

Тест: syntax/ExpectedRedemptionMutationTest.java

6.1. Техника и алатка

Мутациското тестирање внесува мали синтаксни промени (мутанти) во програмата; едно множество тестови „убива“ мутант ако дава различен резултат на мутантот отколку на оригиналот. За разлика од рачно резонирање, тука мутантите се ГЕНЕРИРААТ И ИЗВРШУВААТ РЕАЛНО со PIT (pitest) — стандардна алатка што го мутира бајткодот на PawnContract, ги извршува тестовите врз секој мутант и го пресметува резултатот.

мутациска покриеност = убиени / генерирани

јачина на тестот (test strength) = убиени / (генерирани - непокриени)

Извршување:

```
./mvnw -Ppit test-compile org.pitest:pitest-maven:mutationCoverage
Извештај: target/pit-reports/index.html
pom.xml (профил `pit`): targetClasses=PawnContract,
targetTests=ExpectedRedemptionMutationTest,
excludedMethods=extend,updateTerms,redeem,forfeit,isActive,daysOverdue,ensureActive
```

6.2. Резултат — реален PIT извештај

#	метод	линија	PIT оператор	статус
1	totalDue	L191	NullReturnVals	УБИЕН
2	expectedRedemptionAmount	L201	NegateConditionals	УБИЕН
3	expectedRedemptionAmount	L201	NullReturnVals	УБИЕН
4	isMatured	L209	BooleanFalseReturn	УБИЕН
5	isMatured	L209	BooleanTrueReturn	УБИЕН
6	create	L116	NullReturnVals	УБИЕН

генерирани = 6, убиени = 6 -> мутациска покриеност = 100%, јачина = 100%

Забелешка: стандардните (gregor) PIT оператори не се идентични со концептуалните AOR/ROR/COR/LVR — на пр. totalDue користи Money.add(...) (повик на метод, не примитивен '+'), па PIT го мутира враќањето (NullReturnVals) наместо да го замени операторот. Множеството тестови е мутациски адекватно (100%) врз реалните мутанти што PIT ги генерира за оваа единица.

7. Mockito + JUnit — тестирање на интеракции

Единица под тест:

PawnService (сите јавни операции). Тест: mockito_junit/PawnServiceMockitoTest.java

7.1. Техника и алатки

PawnService е апликациски сервис што ги ОРКЕСТРИРА соработниците (репозиториуми + сервиси од други модули). Затоа се мокираат сите соработници и се верификуваат (а) вратената вредност и (б) интеракциите — кои соработници се повикани, со какви аргументи, во каков редослед.

```
@ExtendWith(MockitoExtension.class)    строго стубирање (неискористен стуб го паѓа тестот)
@Mock ... (10 соработници)             репозиториуми, каса, трансакции, артикли, купувачи,
...
@InjectMocks PawnService                Mockito го гради сервисот со инјектирани мокови

Демонстрирано: when/thenReturn, when/thenThrow, verify, verify(never()),
verifyNoInteractions, times(n), ArgumentCaptor, InOrder, eq()/any() matchers
```

7.2. Мапа на тестови

create_savesContractAndDisburses	зачувува договор; исплаќа главница OUT (InOrder)
extend_recordsInflow	камата + провизија како еден ПРИЛИВ; продолжување
зачувано	
extend_nullFeeIsZero	null провизија -> 0
redeem_fullPayment	откуп + артикл означен; менаџер HE е известен
redeem_underpayment_flagsManager	потплата -> RISK_FLAG (ArgumentCaptor на пораката)
forfeit_listsItemForSale	конфискува + листа за продажба; нема движење пари
update_increasePrincipal_movesCashOut	зголемување -> плаќа разлика OUT
update_decreasePrincipal_movesCashIn	намалување -> враќа разлика IN
update_unchangedPrincipal_noCash	нема промена -> нема касова интеракција
update_notActive_throws	само АКТИВНИ договори (BusinessRuleException)

update_principalChangeWithoutSession_throws	промена на главница бара сесија
update_notifiesManager	менаџерот добива PAWN_UPDATED известување
get_found / get_notFound	враќа / фрла ResourceNotFoundException
list_scopesToVisibleStaff	филтрирање по вработен -> тимот на повикувачот

8. Spring Boot @WebMvcTest — slice тестирање на контролерот

(Дополнителна техника, надвор од материјалот на предметот.)

Единица под тест:

PawnController (сите крајни точки). Тест: web_mvc/PawnControllerWebMvcTest.java

8.1. Техника и алатки

@WebMvcTest е slice тест: подига само веб-слојот на Spring MVC (контролерот, JSON сериализација, врзување на @PathVariable/@RequestBody, Bean валидацијата и @ControllerAdvice), без база или цела апликација. Сервисот е мокиран со @MockitoBean, а барањата се извршуваат во-процес со MockMvc.

Конфигурација (бидејќи тестот е надвор од com.volter):

```
@WebMvcTest(PawnController.class) + @AutoConfigureMockMvc(addFilters=false)
локална @SpringBootTestConfiguration (TestApp) + @Import(PawnController,
GlobalExceptionHandler)
@EnableWebSecurity -> го регистрира AuthenticationPrincipalArgumentResolver, за да се
пазреша @AuthenticationPrincipal StaffPrincipal од SecurityContext (поставен во
@BeforeEach)
```

8.2. Покриени крајни точки

GET	/pawns	list_returnsPagedContracts	-> 200, страничен JSON
GET	/pawns/summary	summary_returnsTotals	-> 200, агрегатни зборови
GET	/pawns/{id}	get_returnsContractJson	-> 200, договор JSON
GET	/pawns/{id}	get_missing_returns404	-> 404 (@ControllerAdvice)
POST	/pawns	create_validBody_returns201	-> 201 Created
POST	/pawns	create_invalidBody_returns400	-> 400 (Bean валидација)
PATCH	/pawns/{id}/contract	updateContract_validBody...	-> 200
POST	/pawns/{id}/extend	extend_validBody_returns200	-> 200, продолжување JSON
POST	/pawns/{id}/redeem	redeem_validBody_returns200	-> 200
POST	/pawns/{id}/forfeit	forfeit_returns200	-> 200 (без тело)

Вкупно 10 теста: сите 8 крајни точки + негативните патеки 400 и 404.

Разлика од Postman тестирањето (§9): Postman вози преку реален HTTP кон целосно подигнат сервер (црна кутија, end-to-end), додека овој slice е во-процес со мокиран сервис и го фиксира HTTP договорот (статус/JSON/валидација) за милисекунди. Тие се комплементарни.

9. API тестирање на PawnController преку Postman

(Дополнителна техника, надвор од материјалот на предметот.)

Единица под тест:

PawnController преку реален HTTP (сите 8 крајни точки).

Артефакт: postman/Volter_Pawn.postman_collection.json

9.1. Техника и алатки

За разлика од @WebMvcTest (во-процес, мокиран сервис), Postman тестирањето е ЦРНА КУТИЈА врз ЦЕЛОСНО подигнатата апликација: барањата патуваат преку вистински HTTP, низ безбедносниот филтер, контролерот, сервисот и базата. Со тоа се проверува целиот договор што го гледа клиентот — автентикација, авторизација (permissions), статусни кодови, JSON тела и валидација.

Алатка: Postman. Испораката е колекција (Collection v2.1.0) што се внесува преку File → Import. Секое барање има pm.test(...) скрипта што го проверува статусот и (каде што има смисла) телото, и запишува ид-а во променливи на колекцијата за да го поврзе следното барање.

9.2. Автентикација (two-step) и променливи

Заложните крајни точки бараат access token врзан за продавница (multi-tenancy). Затоа колекцијата прво тече низ автентикацијата, а токениот се наследува од ниво на колекција (Bearer accessToken).

Auth (по редослед):

1. Login POST /auth/login -> зачувува preAuthToken и shopId (од shops[0])
2. Select Shop POST /auth/select-shop -> Bearer preAuthToken; зачувува accessToken

Сите променливи имаат дефинирани почетни вредности (username/password и seed ид-а), па колекцијата е употреблива веднаш по увоз:

```
baseUrl=http://localhost:8080 username=user password=password
shopId=1 registerId=1 sessionId=5 customerId=1 pawnId=1 pawnIdRedeem=6
pawnIdForfeit=7
```

preAuthToken, accessToken = празни; ги пополнуваат скриптите на Login/Select Shop. Setup ги освежува registerId/sessionId/customerId со НОВОСОЗДАДЕНИ (гарантирано отворена сесија).

9.3. Setup — предуслови (отворена сесија)

Create/extend/redeem бараат ПОСТОЕЧКИ купувач и ОТВОРЕНА касова сесија (парите се движат наспроти неа). Папката Setup ги подготвува и ги зачувува ид-ата во променливи:

Setup (по редослед):

1. Create cash register POST /cash-registers -> registerId
2. Open session POST /cash-registers/{registerId}/open-session -> sessionId (проверува status==OPEN)
3. Create customer POST /customers -> customerId

9.4. Покриени крајни точки (папка Pawns)

GET /pawns филтри/пагинација)	List pawns	-> 200, страничен JSON (+
GET /pawns/summary	Summary	-> 200, агрегатни зборови
GET /pawns/{id}	Get by id	-> 200 (404 ако не постои)
POST /pawns	Create	-> 201; зачувува pawnId (главен договор)
PATCH /pawns/{id}/contract	Update terms	-> 200 (на pawnId)
POST /pawns/{id}/extend	Extend	-> 200 (на pawnId)
POST /pawns	Create (for redeem)	-> 201; зачувува pawnIdRedeem
POST /pawns/{id}/redeem	Redeem	-> 200 (на pawnIdRedeem)
POST /pawns	Create (for forfeit)	-> 201; зачувува pawnIdForfeit
POST /pawns/{id}/forfeit	Forfeit	-> 200 (на pawnIdForfeit, без тело)

Redeem и Forfeit се ТЕРМИНАЛНИ и заемно исклучиви — договорот завршува или како REDEEMED или како FORFEITED, а операцијата бара статус ACTIVE. Затоа секоја од нив

добива СВОЈ свеж договор (pawnIdRedeem, pawnIdForfeit), па целата папка може да се изврши по ред без судир (инаку веќе-откупениот договор не би можел да се конфискува).

9.5. Извршување

- 1) Подигни ја апликацијата: `./mvnw spring-boot:run`
- 2) Postman: File -> Import -> избери `src/test/java/skit_project/tests/postman/Volter_Pawn.postman_collection.json`
- 3) Изврши ги папките по ред: Auth -> Setup -> Pawns
(или сето одеднаш преку Collection Runner; променливите се пренесуваат автоматски).

Постман го покрива истиот контролер како `@WebMvcTest (§8)`, но од надвор и целосно: реален HTTP, вистински сервер и база наспроти брз во-процес slice со мокиран сервис. Двете се комплементарни — slice-от го фиксира договорот за милисекунди, Postman го потврдува вистинскиот систем.

10. Spring Boot end-to-end — интеграциски тест на работен тек

(Дополнителна техника, надвор од материјалот на предметот.)

Работен тек под тест:

СОЗДАВАЊЕ на залог → ПРОДОЛЖУВАЊЕ на залог, преку целиот сервисен слој.

Тест: `workflow/PawnExtensionWorkflowIntegrationTest.java`

10.1. Техника и алатки

За разлика од unit и slice тестовите (со мокови), овој подига ЦЕЛА апликација (`@SpringBootTest`) наспроти РЕАЛЕН PostgreSQL (`Testcontainers`) и го тера работниот тек низ вистинскиот сервисен слој — без мокови. Со тоа се докажува дека соработничките модули (pawn, inventory, customer, cashregister, transaction) навистина работат заедно и запишуваат точна состојба во базата (вклучувајќи ORM мапирања, повеќекратна закупеност и транзакции).

10.2. Работен тек и проверки

Чекор 1 — create : отвора АКТИВЕН договор; исплаќа главница (1000) ОД сесијата (OUT).
Чекор 2 — extend : наплаќа камата (100) ВО сесијата (IN); поместува рок +30 дена; запишува продолжување.

Се тврди:

- статус == ACTIVE; dueDate == issueDate+30 (create), потоа +60 (extend)
- биланс на сесија: 5000 -> 4000 (главница OUT) -> 4100 (камата IN)
- точно ЕДНО ново продолжување (делта: count после == count пред + 1)
- PAWN трансакција со насока IN, износ 100, врзана за нашата сесија

10.3. Детали за поставувањето

- Тестот работи во СВОЈА привремена tenant шема (`schemaInitializer.initializeTenantSchema`; `DROP SCHEMA ... CASCADE` во `@AfterEach`), па секое извршување е изолирано.

- Свежа tenant шема се полни со демо податоци (db/seed/tenant/V2__seed_demo_data.sql), меѓу кои и постоечко продолжување. Затоа тестот користи ДЕЛТА-проверки (не апсолутни count-ови) — робусно на seed податоци.
- Бидејќи класата е надвор од com.volter, @SpringBootTest е насочен кон VolterApplication.

```
Извршување (бапа Docker; е *IntegrationTest -> Failsafe го извршува):
./mvnw verify
или само тој:
./mvnw test-compile failsafe:integration-test -
Dit.test=PawnExtensionWorkflowIntegrationTest
Ако Testcontainers не наоѓа Docker, покажи на веќе-подигнат Postgres преку
IT_DATASOURCE_URL.
```

11. Тестирање базирано на својства (Property-Based)

(Дополнителна техника, надвор од материјалот на предметот.)

Единици под тест:

Money (вредносен објект) и PawnContract (азрегат). Тест:
property_based/PawnPropertyBasedTest.java

11.1. Техника и алатки

Наместо тврдења врз неколку рачно избрани примери, се наведува СВОЈСТВО — правило што мора да важи за секој валиден влез — и се проверува врз многу случајно генерирани влезови. За да остане без нови зависности, се користи @RepeatedTest од JUnit со java.util.Random со фиксно семе (репродуцибилни извршувања). Наменска библиотека (jqwik/QuickTheories) би обезбедила генератори и намалување (shrinking); идејата е идентична.

11.2. Проверени својства

```
Money (износи 0..1_000_000):
P1 add е точна и комутативна
P2 subtract ја инвертира add: (a+b)-b == a
P3 негативен износ секогаш фрла IllegalArgumentException

PawnContract (главница 1.., камата 0.., рок 1..365):
P4 dueDate == issueDate + рок (строго по issueDate); originalDueDate == dueDate
P5 totalDue == главница + камата; свеж договор не е доспеан и expected == totalDue
P6 extend е монотона: секоја ненегативна камата не го поместува рокот наназад и
не го менува originalDueDate; се запишува точно едно продолжување
```

Секое својство се извршува 100-200 пати со различни генерирани влезови.

12. Заклучок

Тестирањето на PawnService и придружните класи е покриено со девет комплементарни техники: четири од материјалот на предметот (ISP/BCC, графови, логичко, мутациско) и пет дополнителни (Mockito/JUnit интеракции, Spring Boot slice, API тестирање преку Postman, Spring Boot end-to-end интеграциски, и тестирање базирано на својства). Мутациското

тестирање се извршува со реални мутанти преку PIT (100% мутациска покриеност), интеграцискиот тест го потврдува целиот работен тек наспроти вистински PostgreSQL, а Postman колекцијата го проверува целосниот HTTP договор на контролерот како црна кутија. Со тоа модулот за залог е покриен од доменската логика до надворешниот API.